Step 1 — Multi Table Data Model

Introduction

In many real-world analytics projects, datasets arrive in a single flat file containing multiple types of entities. While convenient for storage, this structure is inefficient for analytics pipelines.

The dataset used in this project contains information about:

- Customers
- Orders
- Support Interactions
- Agents and Organizational Hierarchy

All these entities are stored in a single denormalized table, which creates problems such as:

- Data redundancy
- Poor query performance
- Difficulty in scaling analytics systems
- Challenges in integrating with BI tools

To address this, we apply database normalization principles and split the dataset into logical relational tables.

Objective of This Step

The goal of this notebook is to transform the raw dataset into multiple structured tables.

These tables will later be stored in a database and used for analytics.

The tables we will create are:

- Customers Table
  - unique_id
  - customer_city
- Orders Table
  - order_id
  - order_date_time
  - item_price
  - product_category

- Agents Table
  - agent_name
  - supervisor
  - manager
  - tenure_bucket
  - agent_shift
- Interactions Table
  - unique_id
  - channel_name
  - category
  - sub_category
  - customer_remarks
  - issue_reported_at
  - issue_responded
  - connected_handling_time
  - csat_score

Expected Outcome

By the end of this notebook we will have:

- Cleaned and standardized dataset
- Removed duplicate records
- Corrected data types
- Created logical relational tables
- Validated table structures

These tables will serve as the foundation for our analytics pipeline.

In [1]:
```
# Import necessary libraries

import pandas as pd
import numpy as np
```

In [2]:
```
# Load dataset

file_path = "data/Customer_support_data.csv"

df = pd.read_csv(file_path)
```

In [3]:
```
# Preview dataset

df.head()
```

Out[3]:

	Unique id	channel_name	category	Sub-category	Customer Remarks	Order_id	o
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Outcall	Product Queries	Life Insurance	NaN	c27c9bb4-fa36-4140-9f1f-21009254ffdb	
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Outcall	Product Queries	Product Specific Information	NaN	d406b0c7-ce17-4654-b9de-f08d421254bd	
2	200814dd-27c7-4149-ba2b-bd3af3092880	Inbound	Order Related	Installation/demo	NaN	c273368d-b961-44cb-beaf-62d6fd6c00d5	
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Inbound	Returns	Reverse Pickup Enquiry	NaN	5aed0059-55a4-4ec6-bb54-97942092020a	
4	ba903143-1e54-406c-b969-46c52f92e5df	Inbound	Cancellation	Not Needed	NaN	e8bed5a9-6933-4aff-9dc6-ccefd7dcde59	

Dataset Inspection

Before performing any transformation, it is important to understand the dataset structure.

In this step we examine:

- Column names
- Data types
- Dataset dimensions
- Missing values

This helps identify potential data quality issues early in the pipeline.

In [4]:
```
# Dataset shape

df.shape
```

Out[4]:
```
(85907, 20)
```

In [5]:
```
# Column names

df.columns
```

Out[5]:
```
Index(['Unique id', 'channel_name', 'category', 'Sub-category',
       'Customer Remarks', 'Order_id', 'order_date_time', 'Issue_reported at',
       'issue_responded', 'Survey_response_Date', 'Customer_City',
       'Product_category', 'Item_price', 'connected_handling_time',
       'Agent_name', 'Supervisor', 'Manager', 'Tenure Bucket', 'Agent Shift',
       'CSAT Score'],
      dtype='object')
```

In [6]:
```
# Dataset information

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 85907 entries, 0 to 85906
Data columns (total 20 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Unique id                85907 non-null  object
 1   channel_name             85907 non-null  object
 2   category                 85907 non-null  object
 3   Sub-category             85907 non-null  object
 4   Customer Remarks         28742 non-null  object
 5   Order_id                 67675 non-null  object
 6   order_date_time          17214 non-null  object
 7   Issue_reported at        85907 non-null  object
 8   issue_responded          85907 non-null  object
 9   Survey_response_Date     85907 non-null  object
 10  Customer_City            17079 non-null  object
 11  Product_category         17196 non-null  object
 12  Item_price               17206 non-null  float64
 13  connected_handling_time  242 non-null    float64
 14  Agent_name               85907 non-null  object
 15  Supervisor               85907 non-null  object
 16  Manager                  85907 non-null  object
 17  Tenure Bucket            85907 non-null  object
 18  Agent Shift              85907 non-null  object
 19  CSAT Score               85907 non-null  int64
dtypes: float64(2), int64(1), object(17)
memory usage: 13.1+ MB
```

In [7]:
```
# Summary statistics

df.describe()
```

Out[7]:

	Item_price	connected_handling_time	CSAT Score
count	17206.000000	242.000000	85907.000000
mean	5660.774846	462.400826	4.242157
std	12825.728411	246.295037	1.378903
min	0.000000	0.000000	1.000000
25%	392.000000	293.000000	4.000000
50%	979.000000	427.000000	5.000000
75%	2699.750000	592.250000	5.000000
max	164999.000000	1986.000000	5.000000

Observations & Insights

After reviewing the dataset structure, the following observations can be made:

1. Column Name Consistency

The dataset contains several column names with spaces and inconsistent formatting such as **"Unique id"**, **"Tenure Bucket"**, **"Agent Shift"**, and **"Sub-category"**.
For easier analysis and better coding practices, these columns should later be standardized (for example converting them to lowercase and replacing spaces with underscores).

2. Data Types

Most columns are currently stored as **object data type**, including columns that represent dates or timestamps:

- `order_date_time`
- `Issue_reported at`
- `issue_responded`
- `Survey_response_Date`

These columns should be converted to **datetime format** to enable time-based analysis such as response time, resolution time, and trend analysis.

3. Missing Values

Several columns contain a significant number of missing values:

- `Customer Remarks` contains many null values, indicating that not every customer leaves feedback.
- `Order_id` , `order_date_time` , `Customer_City` , `Product_category` , and `Item_price` have a large number of missing entries, suggesting that not all customer

interactions are directly related to a specific order.
- `connected_handling_time` has extremely few non-null values compared to the dataset size, which may limit its usefulness for analysis.

These missing values will need to be handled carefully during the data cleaning stage.

4. Numerical Columns

The dataset contains three numerical columns:

- `Item_price`
- `connected_handling_time`
- `CSAT Score`

From the summary statistics:

- The **average CSAT Score is approximately 4.24**, indicating generally positive customer satisfaction.
- `Item_price` shows a wide range between minimum and maximum values, suggesting the presence of both low-cost and high-value products.

5. Dataset Size and Structure

The dataset contains **85,907 rows and 20 columns**, making it a relatively large dataset suitable for advanced analytics.

Overall, the dataset includes a mix of **customer interaction data, agent performance information, product/order data, and customer satisfaction scores**, making it suitable for analyzing **customer support performance, operational efficiency, and drivers of customer satisfaction**.

Column Name Standardization

Datasets collected from operational systems often contain inconsistent column naming patterns.

Common issues include:

- spaces in column names
- mixed capitalization
- special characters

Standardizing column names ensures compatibility with:

- SQL databases

- Python pipelines
- BI tools such as Power BI

We convert all column names to:

- lowercase
- underscore-separated format

In [9]:
```
# Clean column names

df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
    .str.replace("-", "_")
)
```

In [10]:
```
# Verify cleaned column names

df.columns
```

Out[10]:
```
Index(['unique_id', 'channel_name', 'category', 'sub_category',
       'customer_remarks', 'order_id', 'order_date_time', 'issue_reported_at',
       'issue_responded', 'survey_response_date', 'customer_city',
       'product_category', 'item_price', 'connected_handling_time',
       'agent_name', 'supervisor', 'manager', 'tenure_bucket', 'agent_shift',
       'csat_score'],
      dtype='object')
```

Duplicate Record Detection

Duplicate records can significantly distort analytics results.

In customer support systems, duplicates may occur due to:

- system logging errors
- repeated imports
- integration issues

We check for duplicate rows before creating relational tables.

In [11]:
```
# Check duplicates

df.duplicated().sum()
```

Out[11]:
```
np.int64(0)
```

In [12]:
```
# Remove duplicates

df = df.drop_duplicates()
```

Observations & Insights

Confirm that duplicate records have been removed.

If duplicates were present, consider investigating the source of duplication.

Data Type Validation

Correct data types are essential for analytics tasks such as:

- time difference calculations
- statistical analysis
- aggregations
- feature engineering

We convert timestamp columns to datetime format.

In [14]:
```
# Convert date columns to datetime format

date_columns = [
    "order_date_time",
    "issue_reported_at",
    "issue_responded"
]

for col in date_columns:
    df[col] = pd.to_datetime(df[col], errors="coerce", dayfirst=True)
```

In [15]:
```
df.dtypes
```

Out[15]:
```
unique_id                        object
channel_name                     object
category                         object
sub_category                     object
customer_remarks                 object
order_id                         object
order_date_time           datetime64[ns]
issue_reported_at         datetime64[ns]
issue_responded           datetime64[ns]
survey_response_date             object
customer_city                    object
product_category                 object
item_price                      float64
connected_handling_time         float64
agent_name                       object
supervisor                       object
manager                          object
tenure_bucket                    object
agent_shift                      object
csat_score                        int64
dtype: object
```

Observations & Insights

Verify that the following columns are now in datetime format:

- order_date_time
- issue_reported_at
- issue_responded

Datetime conversion enables future calculations such as:

- resolution time
- response delay
- order-to-complaint lag

Creating Logical Relational Tables

To improve analytics workflows, the dataset is split into separate relational tables.

This approach follows database normalization principles.

Benefits include:

- reduced redundancy
- faster queries
- easier integration with BI tools
- scalable analytics pipelines

Customers Table

The Customers table contains customer location information.

This allows us to analyze:

- geographic distribution of complaints
- regional support performance

In [16]:
```
customers = df[[
    "unique_id",
    "customer_city"
]].copy()
```

In [17]:
```
customers.head()
```

Out[17]:

	unique_id	customer_city
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	NaN
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	NaN
2	200814dd-27c7-4149-ba2b-bd3af3092880	NaN
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	NaN
4	ba903143-1e54-406c-b969-46c52f92e5df	NaN

Orders Table

The Orders table contains transaction-level information.

This allows analysis such as:

- product issue trends
- relationship between price and complaints

In [18]:
```
orders = df[[
    "order_id",
    "order_date_time",
    "item_price",
    "product_category"
]].drop_duplicates()
```

In [19]:
```
orders.head()
```

Out[19]:

	order_id	order_date_time	item_price	product_category
0	c27c9bb4-fa36-4140-9f1f-21009254ffdb	NaT	NaN	NaN
1	d406b0c7-ce17-4654-b9de-f08d421254bd	NaT	NaN	NaN
2	c273368d-b961-44cb-beaf-62d6fd6c00d5	NaT	NaN	NaN
3	5aed0059-55a4-4ec6-bb54-97942092020a	NaT	NaN	NaN
4	e8bed5a9-6933-4aff-9dc6-ccefd7dcde59	NaT	NaN	NaN

Agents Table

The Agents table contains support agent information and organizational hierarchy.

This enables analysis such as:

- agent performance comparison
- team-level analytics
- shift productivity

In [20]:

```
agents = df[[
    "agent_name",
    "supervisor",
    "manager",
    "tenure_bucket",
    "agent_shift"
]].drop_duplicates()
```

In [21]:

```
agents.head()
```

Out[21]:

	agent_name	supervisor	manager	tenure_bucket	agent_shift
0	Richard Buchanan	Mason Gupta	Jennifer Nguyen	On Job Training	Morning
1	Vicki Collins	Dylan Kim	Michael Lee	>90	Morning
2	Duane Norman	Jackson Park	William Kim	On Job Training	Evening
3	Patrick Flores	Olivia Wang	John Smith	>90	Evening
4	Christopher Sanchez	Austin Johnson	Michael Lee	0-30	Morning

Interactions Table

The Interactions table represents customer support interactions.

Each row corresponds to a support ticket or interaction.

This table contains:

- issue category
- communication channel
- response timestamps
- handling time
- customer satisfaction score

In [22]:
```
interactions = df[[
    "unique_id",
    "channel_name",
    "category",
    "sub_category",
    "customer_remarks",
    "issue_reported_at",
    "issue_responded",
    "connected_handling_time",
    "csat_score"
]].copy()
```

In [23]:
```
interactions.head()
```

Out[23]:

	unique_id	channel_name	category	sub_category	customer_remarks	issue_rep
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Outcall	Product Queries	Life Insurance	NaN	20
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Outcall	Product Queries	Product Specific Information	NaN	20
2	200814dd-27c7-4149-ba2b-bd3af3092880	Inbound	Order Related	Installation/demo	NaN	20
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Inbound	Returns	Reverse Pickup Enquiry	NaN	20
4	ba903143-1e54-406c-b969-46c52f92e5df	Inbound	Cancellation	Not Needed	NaN	20

Table Validation

After creating relational tables, we validate:

- number of rows
- number of columns
- preview of records

This ensures that tables were created correctly.

In [24]:
```
print("Customers Table Shape:", customers.shape)
print("Orders Table Shape:", orders.shape)
print("Agents Table Shape:", agents.shape)
print("Interactions Table Shape:", interactions.shape)
```

```
Customers Table Shape: (85907, 2)
Orders Table Shape: (67676, 4)
Agents Table Shape: (1371, 5)
Interactions Table Shape: (85907, 9)
```

Observations & Insights

Review the shapes and previews of the tables.

Ensure that:

- each table represents a unique entity
- duplicates were removed correctly
- column selection matches the intended data model

These normalized tables will be used in the next step to build the data ingestion pipeline and SQLite database.

Exporting Normalized Tables

Why Export the Tables?

After completing the data cleaning and relational modeling process, the next step in the analytics pipeline is to store the normalized tables as intermediate datasets.

In production analytics systems, cleaned datasets are often stored as intermediate data layers before being loaded into a database or data warehouse.

Exporting the tables provides several benefits:

- Preserves cleaned data for reproducibility
- Separates raw data from transformed data
- Enables modular pipeline development
- Allows the ingestion pipeline to load structured data directly

These exported tables will serve as the input for the data ingestion pipeline in the next notebook, where they will be loaded into a SQLite database.

Tables Being Exported

The following normalized tables will be saved:

- Customers Table
  - unique_id
  - customer_city
- Orders Table
  - order_id
  - order_date_time
  - item_price
  - product_category
- Agents Table
  - agent_name
  - supervisor
  - manager
  - tenure_bucket
  - agent_shift
- Interactions Table
  - unique_id
  - channel_name
  - category
  - sub_category
  - customer_remarks
  - issue_reported_at
  - issue_responded
  - connected_handling_time
  - csat_score

These tables will be stored inside the data directory and used by the next stage of the analytics workflow.

In [26]:
```
# Export normalized tables for the ingestion pipeline

customers.to_csv("Data/customers.csv", index=False)
orders.to_csv("Data/orders.csv", index=False)
agents.to_csv("Data/agents.csv", index=False)
interactions.to_csv("Data/interactions.csv", index=False)
```

Observations & Insights

The normalized tables have now been successfully exported as CSV files.

These files represent the cleaned and structured data layer of the project and will be used in the next stage of the analytics pipeline.

At this point we have achieved the following:

- cleaned and standardized the raw dataset
- removed duplicate records
- validated data types
- created relational tables
- exported normalized datasets for ingestion

These exported tables will now be used to build a data ingestion pipeline into a SQLite database in the next notebook.

This modular workflow mirrors real-world data engineering practices where datasets move through multiple structured layers before analysis and reporting.

Outcome

This notebook completed the data modeling stage of the analytics pipeline.

Key accomplishments include:

- understanding the dataset structure
- cleaning and standardizing the data
- validating and correcting data types
- removing duplicate records
- creating normalized relational tables
- exporting structured datasets for downstream processing

These steps form the foundation of the analytics system and ensure that the dataset is reliable, structured, and ready for database ingestion.

In the next notebook we will build a data ingestion pipeline that loads these tables into a SQLite database, enabling SQL queries, feature engineering, and advanced analytics.

In [ ]:

```
```